UNIVERSITY OF THE WESTERN CAPE

Computer Science 211
Practical 5

21 February 2014

Theme: Classes, Stacks, Linked lists and Generics

---

1. Create a data structure known as a stack. As in the case of a stack of dishes, the last one placed on the stack is the first to be removed. So items are added and removed from the top of the stack, and not from the bottom. First, create a LinkedListInteger class that implements the interface that appears below. Note, your linked list must store Integer objects not primitive integers.

```
public interface SLinkedListInteger{
        public void displayList();          // displays the data on the list
        public int length();                // returns the number of nodes on the list
        public void addANodeToStart(Integer itemToAdd); // adds a node in front of the
                                                        // first node
        public SNodeInteger deleteHeadNode();           // deletes the first node on the list
        public boolean isOnList(Integer dataItem); // returns a reference to the first node
                                                   // containing the integer dataItem; returns
                                                   // null if dataItem is not on the list
}
```

Secondly, create a class, **LinkedStackInteger** that stores Integer objects. Your LinkedStackInteger class should have the following private attribute,

top – a reference to the last node placed on the stack

and the following methods;

getHeight() – an int type method that returns the number of elements stored on the stack

isEmpty() – a boolean type method that returns true if the stack is empty, else false is returned

push(element) – a 'void method', that places the Integer object on the stack

pop() – an Integer type method that removes the top element from the stack
(NOTE : An element can only be removed from a non-empty stack)

head() – an Integer type method that reveals the top element on the stack without removing it from the stack.

display() – a void method, that displays the contents of the stack from the **top element downwards** to the first element placed on the stack.

Note, the different references to int and Integer data types above.
Ensure that you have coded the following classes, SNodeInteger, SLinkedListInteger, and LinkedStackInteger, where SNodeInteger stores an Integer object and not a primitive data type, int. Note, the integer indicating the position of a parenthesis to an Integer object must first be converted to an Integer object before storing it onto the stack. Also, to access the value of the Integer object use the appropriate method of the Integer wrapper class.

Develop a **driver** to test your class, i.e. write a menu driven program that allows the user to select and apply any of the 6 operations, described above, to an Integer stack. Save your program as **TesterStackOfIntegers**.java.

2. Modify all relevant classes using generics so that *any reference type* can be used on the stack. Observe the disadvantages when not using generics in your programs. Save the updated classes as **SNode**, **SLinkedList** and **LinkedStack**. Using instances of the Integer wrapper class and retest your classes. Test your updated classes as well as the **TesterStackGenerics** class.

3. Develop a program that will match the left and right parenthesis of any algebraic expression. An expression containing parenthesis is entered as a string. The program should then match all the left parentheses with the right parentheses that appear in the expression. Any unmatched left or right parenthesis should be reported by the program. Use an object of the class **LinkedStack** developed in question 2 in your implementation. The algorithm to match parentheses appears below.

MatchingParenthesis

<table>
<tr><td colspan="4">Initialize string strng to parenthesized expression</td></tr>
<tr><td></td><td colspan="3">Repeat indx=0 to length of strng -1</td></tr>
<tr><td></td><td colspan="3">Extract character at position index of strng and store in variable chr; chr</td></tr>
<tr><td></td><td>\T chr = “(” ? /F</td><td colspan="2"></td></tr>
<tr><td></td><td rowspan="3">push position of chr onto stack, i.e index in Integer form</td><td colspan="2">\T chr = “)”? /F</td></tr>
<tr><td></td><td colspan="2">\T Stack not empty? /F</td></tr>
<tr><td></td><td>pop Integer from stack</td><td>Display unmatched right parenthesis at position indx</td></tr>
<tr><td></td><td colspan="3">While stack is not empty</td></tr>
<tr><td></td><td colspan="3">Pop integer from stack and store in posn
Display unmatched left parentheses at position posn</td></tr>
</table>

Your program should implement this algorithm. Save your program as **MatchParenthesis1.java**.

```
Sample input:    (((a+b)c)d)e)f)g)h(((j+k)*l)/m
                 012345678901234567890123456789
                           1         2         3
 Sample output:
            unmatched right parenthesis in position 12
            unmatched right parenthesis in position 14
            unmatched right parenthesis in position 16
            unmatched left parenthesis in position  18
```

4. Modify the program developed in 3 so that the unmatched parentheses are marked on the next line of output using characters “<” and “>” as indicated in the sample output below. The values of indices of the string are also displayed to enable you to verify the correctness of the output. Look at the sample output below.

```
Sample input:
     (((a+b)c)d)e)f)g)h(((j+k)*l)/m

Sample output
     (((a+b)c)d)e)f)g)h(((j+k)*l)/m
                      < < < >
          012345678901234567890123456789
                    1         2         3
```

Save your program as **MatchParenthesis2.java**

----------END OF SHEET----------